Artificial Neural Networks

Nicole DePergola

April 2024

1 Introduction

Artificial Neural Networks (ANNs) have emerged as a powerful tool for various machine learning tasks. Among the plethora of applications, the classification of handwritten digits stands as a quintessential benchmark for evaluating the efficacy of ANNs. I will explain the fundamental principles and practical applications of Artificial Neural Networks, with a specific focus on their utilization in classifying handwritten digits using the MNIST dataset. MNIST, a widely-used dataset comprising 28x28 pixel images of handwritten digits ranging from 0 to 9, serves as an exemplary platform for training and evaluating the performance of neural network models. I will also do an investigation into two popular methods of running ANNs, and compare their performances on the MNIST dataset.

2 Methods & Results

2.1 Background

Artificial Neural Networks (ANNs) are computational models inspired by the structure and function of biological neural networks in the human brain. ANNs are composed of interconnected nodes, called neurons, organized in layers. Each neuron receives input signals, processes them, and produces an output signal. The process of how neural networks work involves several key components.

Firstly, during the forward pass, input data is passed through the network, with each neuron in a layer computing a weighted sum of its inputs and applying an activation function to produce an output. The weighted sum of inputs is computed by multiplying each input by its corresponding weight, summing these products, and adding a bias term. The resulting value is then passed through an activation function, which introduces non-linearity into the network and allows it to learn complex patterns and relationships in the data. Common activation functions include sigmoid, ReLU (Rectified Linear Unit), and softmax. Sigmoid and ReLu are used in the hidden layer(s) due to their simplicity and effectiveness, while the softmax function is used in the output layer to produce probability distributions over the classes (in my case the classes are each digit 0-9).

Next, the network is trained using a process called backpropagation combined with gradient descent. Backpropagation involves iteratively adjusting the weights of the connections

to minimize the difference between the predicted outputs and the true outputs (labels) of the training data. This process begins with the calculation of the loss function, which measures the difference between the predicted outputs and the true outputs. Common loss functions include mean squared error (MSE) for regression tasks and categorical cross-entropy for classification tasks. The gradients of the loss function with respect to the weights are then computed using the chain rule of calculus, starting from the output layer and propagating backward through the network. The weights are updated in the direction that reduces the loss, typically using optimization algorithms like gradient descent. Gradient descent iteratively updates the weights by taking small steps in the direction of the negative gradient of the loss function, gradually minimizing the loss and improving the model's performance. See Figure 1 for a visual of an ANN.

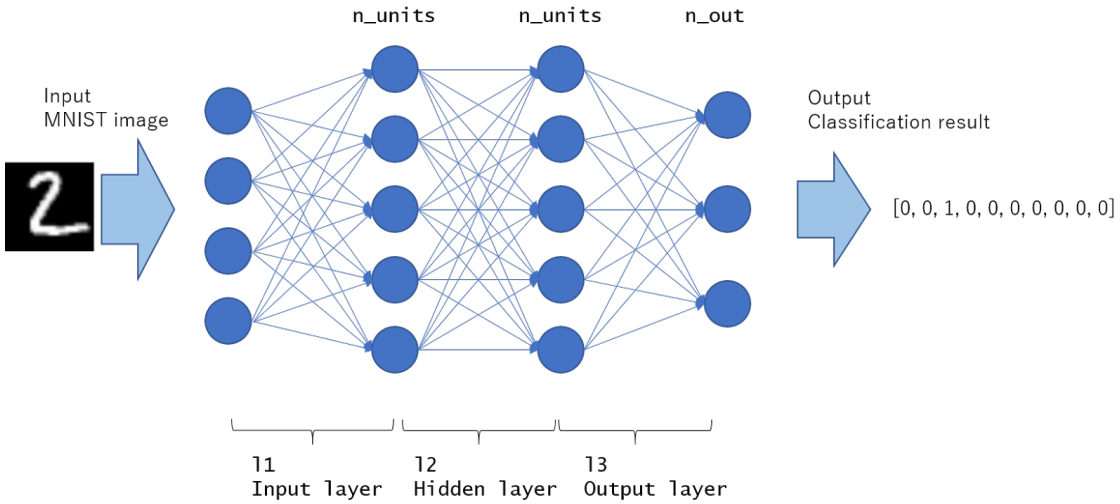


Figure 1: Structure of Artificial Neural Network (ANN).

In the context of training an ANN to recognize numbers, we implement the dataset of written digits collected by the National Institute of Standards and Technology (NIST). I used a modified version of this dataset, MNIST, to train an ANN. This exercise is kind of like the "Hello World" of machine learning.

After I establish my first ANN, I want to investigate the differences between using the ReLu method and the sigmoid method on my hidden layers. Some deep-learning researchers have cited some problems with the sigmoid method, as it becomes super flat at the extremes, when the weighted sum being passed in has a large magnitude (See Equation 1).

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

See how the sigmoid $\sigma(x)$ is close to 0 when x is a large negative number, and is close to 1 when x is a large positive number.

The process of training the neural network essentially boils down to wiggling the values of all the weights and biases and watching what happens; when a little wiggle to a weight is effective, the network does more of that, and when it isn't helpful, the network does the

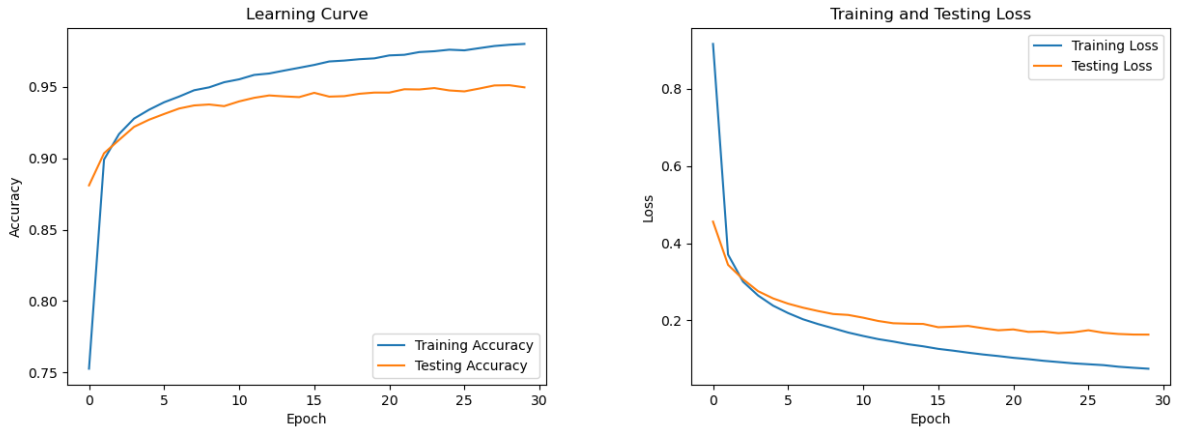
opposite. Since the sigmoid function gets so flat, wiggling the weights doesn't really do much of anything, which means that the process of training the network takes a long time.

Because of this, people now tend to use a function called ReLU (Rectified Linear Unit) instead. ReLU returns a 0 for any negative input, and doesn't change the positive inputs at all. So, unlike sigmoid, the output of ReLU never flattens out, no matter how large the activation weighted sum becomes. So wiggling the weights always gives useful feedback about how the network should change. This makes the training process much faster and more efficient, especially as there are more layers involved. Since I am not creating extremely large networks, I will see the results of using each method.

2.2 Training a Network with the MNIST Dataset

To build my first Artificial Neural Network (ANN) I began by importing the MNIST dataset using the Python package *Keras*. *Keras* includes all necessary functions to handle the data and create ANNs. After loading in the data, I began building the network, consisting of one hidden layer with 30 neurons, and the output layer with 10 neurons (one for each digit, 0-9). For this network, I defined my hidden layer to use the ReLu method for the activation function. Next, I selected 30 subsets of the MNIST data to be used to train this network. I set the epochs (the number of times the dataset will be passed forward and backward through the neural network during training) to 30, and set aside 20% of the sets to be used as testing, or validation, data. Testing data is not used for training. After training, we introduce fresh data (the testing data) to the network to see how it performs on digits its never seen before.

Figure 2 shows the learning curve and the loss of this network. During training, it's evident that the network became very good at identifying the training set of digits, finishing with an accuracy of 0.977. But it exhibited lower accuracy with the testing set of digits that it was not trained on, with the validation accuracy ending at 0.9497.



(a) Learning curve over time.

(b) Loss of network over time.

Figure 2: Artificial Neural Network with one hidden layer performance.

Figure 6b shows the loss for both sets of data run through the network. The training set eventually reached a loss value of 0.0867 which indicates good performance in the network. The testing set finished with a loss value of 0.1634 in this network.

Figure 3 shows the frequency of misclassified digits. 5, 8, and 9 are leading the pack with the most misclassifications, meaning that this network incorrectly identified those digits the most. 0, 1, and 6 have the least misclassfications, so the network was the best at identifying those digits.

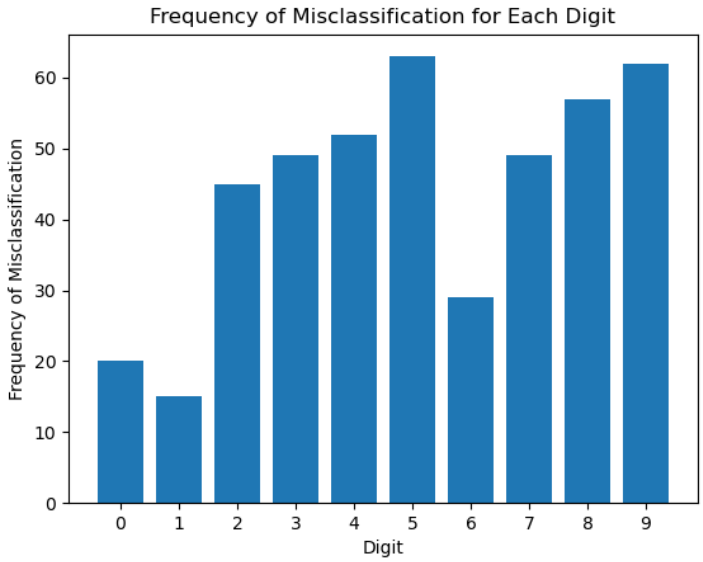


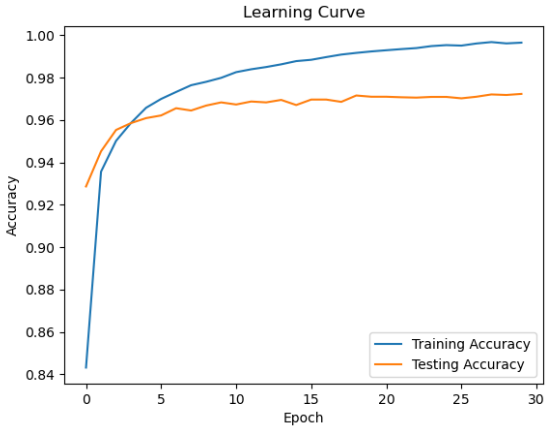
Figure 3: Histogram of misidentified digits during validation run.

2.3 Bigger Networks

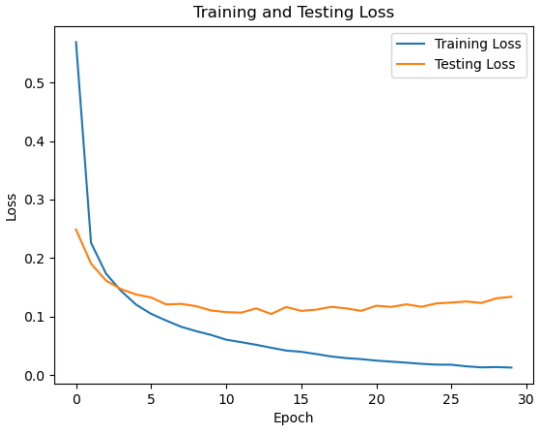
I wanted my computer to work a bit harder training these networks, so I made them bigger and also started an investigation into the different activation function methods. I added a second hidden layer to my network, and increased the number of neurons in each hidden layer to 50. I also increased my training subset to 50 sets. Figure 4 shows the learning curve and loss over time of this bigger network. We see an increase in accuracy for both training and testing sets compared to the first, smaller network. As well as smaller loss values for both sets compared to the previous network. Table 1 shows the performance of this network.

Training Accuracy	Training Loss	Testing Accuracy	Testing Loss
0.9926	0.0350	0.9723	0.1338

Table 1: Performance statistics for ReLu ANN.



(a) Learning curve over time.



(b) Loss of network over time.

Figure 4: Artificial Neural Network performance using ReLu activation function.

Figure 5 shows the misclassfication frequencies for this network. There are considerable less misidentifications than the previous network, which is a good sign. 5, 8, and 9 are still leading the pack with the most misclassifications, while 0, 1, 2, and 4 have the least.

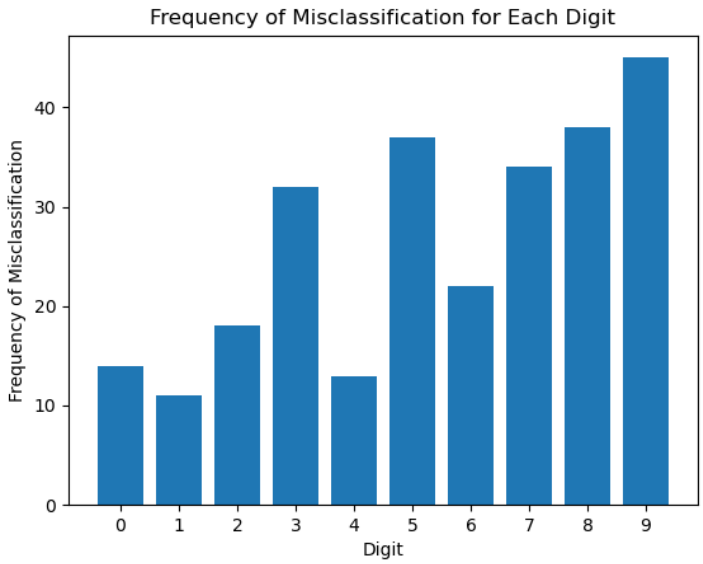
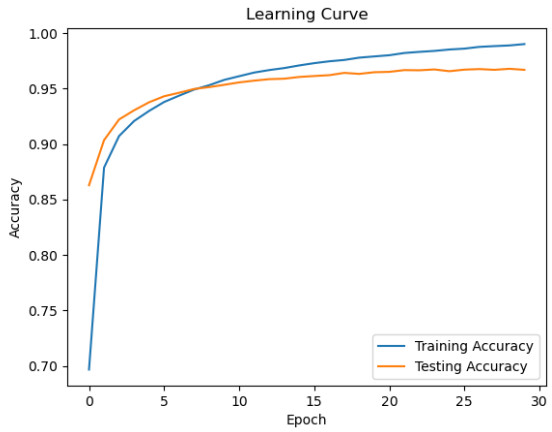
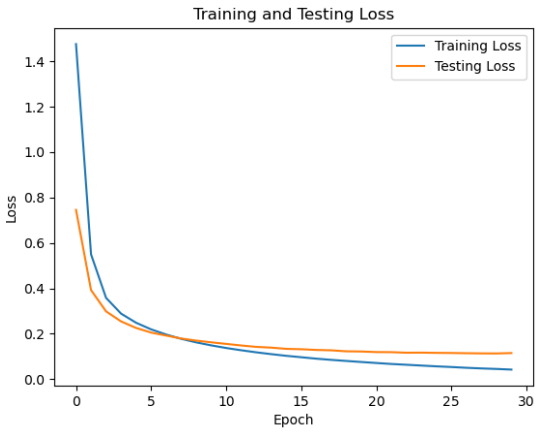


Figure 5: Histogram of misidentified digits for ReLu ANN.

Next, I kept the size of the network and all parameters exactly the same, but I defined the activation functions for the hidden layers to be sigmoid, instead of ReLu. Figure 6 shows the performance of this network. Immediately we see that the curves for both learning and loss are closer than that of the ReLu results. However, we don't quite reach the level of accuracy that the previous network did.



(a) Learning curve over time.



(b) Loss of network over time.

Figure 6: Artificial Neural Network with one hidden layer performance.

Table 2 shows the exact values of performance.

Training Accuracy	Training Loss	Testing Accuracy	Testing Loss
0.9861	0.0537	0.9669	0.1146

Table 2: Performance statistics for sigmoid ANN.

Figure 7 shows the misclassifation frequency for this network. The results are comparable to the ReLu network. I do a more thorough investigation into this next.

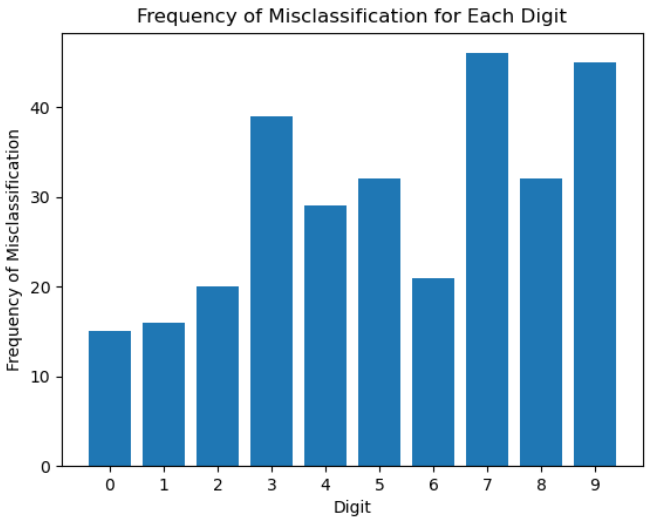
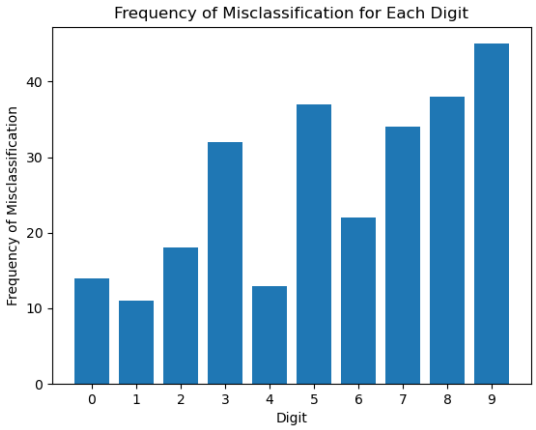


Figure 7: Histogram of misidentified digits for sigmoid ANN.

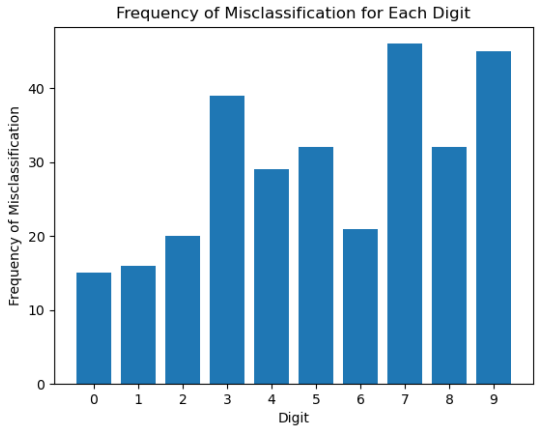
2.4 Investigating Stability

The histograms of misclassified digits that I've been including for each network is very interesting. I commonly see certain numbers being misidentified more than others, whereas numbers like 0 and 1 always have the least misclassifications. I wanted to find more reliable answers into which numbers are hardest to identify, so I created a loop that would train 50 ANNs. I did this for both the ReLu and sigmoid methods. After each network is trained, I stored the information for what digits were misclassified in the validations runs. So, after training 50 sets (in each activation function method) and gathering the misidentified digit data, I averaged the totals and found out the average frequency of misclassified digits. Figure 8 shows the results from both methods.

It would appear that the network using ReLu has the hardest time identifying the digits 5, 8 and 9 (Figure 8a). The network using sigmoid has the hardest time classifying the digits 3, 7 and 9 (Figure 8b). Also something interesting to note: I included a timer in my code for these histograms. It took my computer 10 minutes and 56 seconds to train 50 ANNs using ReLu, but only 10 minutes and 33 seconds to train the same network using sigmoid! ReLu is producing more accurate results for my current configurations, but sigmoid is still slightly faster.



(a) ReLu activation function ANN.



(b) Sigmoid activation function ANN.

Figure 8: Average misclassified digit frequencies for two ANNs.

3 Discussion

I loved this! Thank you for the extension, it really gave me the time to learn how this machine learning technique works and play around with it. I actually enjoyed learning about neural networks, and writing this report wasn't too bad after I had a good understanding of the methods and the code. I'd certainly be interested in trying to make an ANN for something else, maybe pictures of birds to help identify what I see around campus or at home. I also have ambitions to become a software engineer, so I'm glad I got to learn a bit about machine learning; I feel that it's such a growing field and it would be wise to keep up.